SkyLock — Interview Q&A

20 questions on the Flight Booking System backend, from beginner to advanced — covering database design, concurrency, Redis, transactions, and deployment.

Beginner Level

Q1. What does this project do, at a high level?

Answer:
SkyLock is a flight booking backend built with FastAPI. It lets users register, log in, search flights, select and book seats, receive a PDF ticket and email confirmation, and cancel bookings for a refund. The main engineering focus wasn't the UI — it was making sure the booking process behaves correctly even when many users try to book at the same time.

Q2. Why did you choose FastAPI over something like Flask or Django?

Answer:
FastAPI is built on Starlette and supports native **async/await**, which matters a lot for an I/O-heavy app like this — every request involves waiting on a database call, a Redis call, or an external API (Resend for email). Async lets the server handle many concurrent requests without blocking a thread per request. FastAPI also gives automatic request validation via Pydantic and free interactive API docs (Swagger UI at /docs), which sped up development and testing significantly.

Q3. Walk me through your database schema.

Answer:
There are seven tables: **users**, **flights**, **seats**, **bookings**, **payments**, **passengers**, and **refunds**. A flight has many seats. A booking links a user, a flight, and exactly one seat. Each booking has one payment and, optionally, one passenger and one refund. I used foreign keys everywhere so the database itself rejects invalid references (e.g. a seat pointing to a flight that doesn't exist), rather than relying only on application code to catch that.

Q4. What's the difference between a Booking, a Payment, and a Passenger in your schema?

Answer:
A **Booking** is the core record — it links a user to a specific seat on a specific flight and tracks status (pending/confirmed/cancelled). A **Payment** is a separate table tracking the amount charged and its status (success/refunded) — I split this out rather than adding columns to Booking because payment has its own lifecycle (e.g. it can later be refunded independently). A **Passenger** holds the actual traveller's details (name, age, gender, meal preference) — this is separate from the User account because the person travelling isn't always the person who created the account (e.g. booking for a family member).

Q5. How does user authentication work in this project?

Answer:

Users register with an email and password. The password is never stored in plain text — it's hashed using bcrypt via the passlib library before being saved. On login, the submitted password is hashed and compared against the stored hash. If it matches, the server issues a JWT (JSON Web Token) containing the user's identity, signed with a secret key. The frontend then sends this token in the Authorization header on every subsequent request, and a FastAPI dependency (get_current_user) decodes and verifies it before allowing access to protected routes.

Q6. Why hash passwords instead of encrypting them?

Answer:
Encryption is reversible — if someone got the encryption key, they could recover the original password. Hashing (with bcrypt) is a one-way function: there's no way to reverse a hash back into the original password. Bcrypt also automatically applies a random salt per password and is deliberately slow, which makes brute-force and rainbow-table attacks impractical even if the database itself is ever leaked.

Q7. What is CORS, and why did you need to configure it?

Answer:
CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a webpage from making requests to a different domain than the one it was served from, unless that domain explicitly allows it. Since my frontend (skylock-frontend-m3fw.onrender.com) and backend (skylock-api-24gd.onrender.com) are on different subdomains, the browser would block every API call by default. I configured FastAPI's CORSMiddleware to explicitly allow requests from my frontend's exact origin, which tells the browser it's safe to let those requests through.

Intermediate Level

Q8. What is a race condition, and where could one occur in a booking system?

Answer:
A race condition happens when two operations that depend on shared data run at nearly the same time, and the final outcome depends on unpredictable timing. In a booking system, the classic case is: two users both view seat 12A as available, both start booking it at the same moment, and without protection, both bookings could be written to the database — resulting in one seat being sold twice. This is exactly the problem my locking system is designed to prevent.

Q9. Explain your two-layer seat locking system in detail.

Answer:
The first layer is a **Redis lock**, acquired the moment a user selects a seat (before they even finish entering passenger details). This lock has a short TTL (time-to-live) and is keyed by flight and seat ID, storing the ID of the user who holds it. If a second user tries to select the same seat while it's locked, their request is rejected immediately with a 409 Conflict. The second layer is a **database row lock** using SQL's SELECT ... FOR UPDATE, applied at the moment the booking is actually confirmed. This locks that specific seat row in Postgres so no other transaction can read-and-modify it concurrently, and it re-checks that the seat is still unbooked right before committing. The Redis lock is the fast, user-facing first line of defense; the database lock is the authoritative, final guarantee — even if the Redis lock somehow failed or expired, the database would still

refuse to double-book the seat.

Q10. Why do you need both Redis locking and database locking? Isn't one enough?

Answer:
They solve different problems. The Redis lock exists mainly for **user experience** — it gives instant feedback ('this seat is taken') without hitting the database, and it holds the seat during the multi-step process of entering passenger info and confirming. But Redis locks can theoretically expire or be bypassed by a bug. The database lock is the **real guarantee** — it's ACID-compliant, and Postgres itself will not allow two transactions to both successfully update the same locked row. If I only had the Redis lock, a timing edge case or Redis failure could allow a double-booking. If I only had the database lock, users would get a fast 'seat taken' response, but Nothing would hold the seat between selecting it and actually confirming, so the seat could be pulled away mid-checkout.

Q11. What does SELECT ... FOR UPDATE actually do?

Answer:
It's a SQL clause that locks the selected row(s) for the duration of the current transaction. Any other transaction that tries to SELECT ... FOR UPDATE (or otherwise modify) the same row will be forced to wait until the first transaction commits or rolls back. In my confirm_booking function, I run this on the specific seat row before checking is_booked and before creating the booking — so even if two confirm requests arrive at almost the same instant, Postgres serializes them: the first to acquire the lock proceeds and marks the seat booked; the second waits, then sees is_booked = True once it gets the lock, and is rejected with a 409.

Q12. What is an atomic transaction, and why does it matter for your booking flow?

Answer:
Atomic means 'all or nothing' — every part of a transaction succeeds together, or none of it does. In confirm_booking, creating the Booking, the Payment, and the Passenger, and marking the seat as booked, all happen within a single database transaction. If any step fails (say, an IntegrityError from a unique constraint), the entire transaction is rolled back — so I never end up with an orphaned payment record with no matching booking, or a booked seat with no payment on file. This matters enormously for anything involving money or inventory, since partial writes create real data corruption, not just a cosmetic bug.

Q13. What's the purpose of the try/except around IntegrityError in your booking confirmation?

Answer:
It's a defense-in-depth safety net. My application logic already checks is_booked before attempting to book a seat, and the SELECT ... FOR UPDATE row lock prevents concurrent conflicts at the database level. But UniqueConstraint('flight_id', 'seat_number') on the Seat table and the unique constraint on Booking.seat_id are the database's own final guarantee. If, for any unforeseen reason (a bug, a race I didn't anticipate, manual data inconsistency), two inserts still collided, Postgres itself would reject the second one with an IntegrityError. Catching that and returning a clean 409 response, instead of letting it surface as an ugly 500 error, means the system fails safely and predictably even in scenarios I didn't explicitly design for.

Q14. How does Redis caching work in your flight search feature, and why use it?

Answer:
When a user searches flights (by origin, destination, date, etc.), I build a cache key from those exact search parameters. Before querying Postgres, I check Redis for a cached result under that key. If found, I return it immediately, skipping the database entirely. If not, I query Postgres, then store the serialized result in Redis before returning it. This matters because flight searches are read-heavy and often repeated (many users searching Delhi to Mumbai on the same date) — serving those from Redis, which is an in-memory store, is drastically faster than re-running the same SQL query against Postgres every single time, and it reduces load on the database under high traffic.

Q15. How do you keep the cache from serving stale data after a flight is added?

Answer:
I explicitly invalidate the flights cache whenever a new flight is created (in the create_flight endpoint, right after committing to the database). This clears out the cached search results so the next search re-queries Postgres and picks up the new flight, rather than silently returning outdated results. This is a simple write-through invalidation strategy — it trades a small amount of cache-miss cost after writes for guaranteed correctness, which is the right tradeoff for something like flight listings where stale data (showing a flight that no longer exists, or hiding one that does) is a real user-facing problem.

Advanced Level

Q16. Why use selectinload() instead of just accessing relationships directly?

Answer:
By default, SQLAlchemy relationships are lazily loaded — accessing booking.payment triggers a separate query only when you access it. In an async context, this is a real problem: lazy loading requires a synchronous round-trip to the database that doesn't play well with asyncio, and it often fails outright outside of a greenlet context, or silently causes the N+1 query problem (one query per row instead of one query total). selectinload() tells SQLAlchemy to eagerly fetch that relationship in a single additional query up front, as part of the same async-safe call. So when I return a list of bookings with their payment, refund, passenger, and flight data, I fetch all of that in a small, fixed number of queries instead of one query per relationship per row.

Q17. What happens end-to-end when two users try to book the same seat at literally the same millisecond?

Answer:
Say both users already passed the Redis lock stage somehow (e.g. simulating a worst case where Redis locking wasn't in play, or both requests reach confirm_booking directly). Both requests call SELECT seat FOR UPDATE on the same seat row. Postgres allows only one of them to acquire the row lock first; the second request blocks and waits. The first transaction checks is_booked (false), creates the Booking/Payment/Passenger, sets is_booked = True, and commits — releasing the lock. The second transaction then acquires the lock, re-reads the row, sees is_booked is now True, and my code raises a 409 Conflict instead of proceeding. So only one booking is ever created, and the second user gets an immediate, clear 'seat already booked' response rather than a corrupted or duplicate booking. I verified this exact scenario with a script (concurrency_test.py) that fires both requests concurrently using asyncio.gather().

Q18. How does your group booking flow differ from a single seat booking, and what extra complexity does it introduce?

Answer:

Group booking (confirm-group) needs to lock and book multiple seats atomically as one unit — either all seats in the group get booked together, or none do. I loop through every requested seat_id and apply SELECT ... FOR UPDATE to each one before creating any records, so I'm holding row locks on all the relevant seats simultaneously within one transaction. If any seat in the group turns out to be already booked, I roll back the entire transaction and release all previously acquired Redis locks for that group — so a family of four never ends up with three confirmed seats and one silently failed. The extra complexity versus single booking is mainly in this all-or-nothing group semantics and cleaning up partial locks correctly on failure.

Q19. What deployment issue did you actually run into, and how did you diagnose and fix it?

Answer:

When I deployed the backend on Render, booking emails silently weren't being sent, even though bookings succeeded. There were no errors on the client side and the booking flow completed normally, so at first this looked like a bug in my email-sending code. I added more detailed logging around the SMTP call and found the real cause in the logs: Render's free tier blocks outbound traffic on SMTP ports 25, 465, and 587 as an anti-spam measure, so my Gmail SMTP calls were being silently dropped at the network level, not failing due to bad code or credentials. Since I didn't want to pay for a higher tier just to unblock SMTP, I rewrote the email service to use Resend's HTTP API instead of raw SMTP — this sends email over standard HTTPS (port 443), which isn't restricted, so it works fine on the free tier. This also turned out to be a better practice generally: HTTP-based transactional email APIs are what most production systems use over raw SMTP anyway, since they offer better deliverability and don't require managing SMTP credentials directly.

Q20. If this system needed to handle 10x or 100x more traffic, what would you change first?

Answer:

A few things, roughly in priority order. First, I'd move seat availability checks fully onto Redis (rather than only using it as a soft lock before falling back to Postgres row locks), so the database isn't the bottleneck for read-heavy seat-map views under high concurrency. Second, I'd add read replicas for Postgres, since flight search and seat map viewing are read-heavy and could be served from replicas while writes (bookings, payments) go to the primary. Third, I'd move background email/PDF work from FastAPI's built-in BackgroundTasks to a proper task queue like Celery or a dedicated worker service, since BackgroundTasks run in the same process as the web server and could compete for resources under heavy load, whereas a separate queue can scale independently. Finally, I'd look at connection pooling limits — my current pool_size and max_overflow settings are tuned for a small deployment, and would need to be reassessed against actual concurrent connection counts under real load testing, alongside possibly using PgBouncer in front of Postgres to handle many more concurrent connections efficiently.

Tip: for questions 8–13 (concurrency/transactions) and 16–20 (advanced), be ready to talk through the concurrency_test.py script directly — being able to point at a real test that proves the behavior is far stronger than describing it in the abstract.